MC970 - Introdução à programação paralela
Relatório - Exercícios com threads

Daniel de Sousa Cipriano RA: 233228

I. Descrição de sistema

Os exercícios foram executados em um sistema Linux baseado em Debian com a seguinte configuração:

Componente	Descrição
Sistema operacional	MX Linux 64 bits
Processador	AMD Ryzen 5 5500U com SMT
Núcleos	6 físicos, 12 lógicos
Clock	4.05 GHz
Memória RAM	16 GB DDR4

II. Ferramentas

As principais ferramentas utilizadas para desenvolvimento foram:

Hyperfine: gerador de benchmark. Recebe dois comandos e retorna dados comparativos de desempenho entre eles[1]. Foi utilizada a biblioteca `matplotlib` do Python, em conjunto com benchmarks em formato JSON do Hyperfine, para gerar dados e plotar gráficos de speedup por número de threads, eficiência por número de threads e tempo de execução serial/paralelo por número de threads.

Perf: ferramenta utilizada para analisar métricas de desempenho. Nesse contexto foi utilizada para analisar falhas de cache.

Gprof: ferramenta utilizada principalmente para análise individual de desempenho entre funções de programas. Em alguns casos foi integrada ao script `gprof2dot.py`[2] para gerar um grafo em que se pode analisar visualmente o overhead em cada função.

pthread: biblioteca em C voltada à manipulação de threads. As principais funções utilizadas foram `pthread_create()` que tenta criar uma thread, retorna zero caso obtenha sucesso ou diferente de zero caso haja algum erro; `pthread_join()` garante que o programa esperará o encerramento da thread para prosseguir, recebe uma thread e um ponteiro que irá armazenar o retorno da função de thread. Além dessas funções, foram utilizados tipos definidos na biblioteca `pthread` e mutexes para casos de exclusão mútua.

thread: biblioteca em C++ voltada à manipulação de threads. Semelhante à pthread em C, mas com orientação a objetos.

III. hello-threads

A. Objetivo

O objetivo desse exercício foi paralelizar um código simples em C, que imprime em um loop a string ”Hello World”. Foram compilados dois programas, `hello_threads.c` que executa de forma serial e `hello_threads_solution.c` que executa de forma paralela.

B. Resultados

A solução paralelizada foi implementada em `hello_threads_solution.c`. Utilizando Hyperfine para analisar comparativamente os tempos de execução com doze threads, tem-se que a versão paralelizada possui latência média de $(3,004 \pm 0,000)$ s. Enquanto a versão serial possui latência média de $(36,005 \pm 0,001)$ s. O principal ponto de overhead da versão serial é a função printHello(), que é executada em loop. Portanto, ao dividir o trabalho entre threads, nota-se uma diminuição significativa no tempo de execução. Com isso, obtém-se o speedup de $(11,99 \pm 0,00)$. Pode-se notar

que o tempo de execução paralelizada é muito próximo do ideal, pois vale aproximadamente a divisão exata do tempo de execução serializada pelo número de threads. Podemos visualizar a repetição desse comportamento para diferentes números de threads, como mostra a Figura 1. Analisando o gráfico de eficiência por variação de threads da Figura 2, vemos o decrescimento da eficiência com o aumento das threads. Esse comportamento reflete o overhead gerado pela paralelização com gerenciamento de threads e pela concorrência criada quando há mais threads do que núcleos no sistema. Analisando o gráfico de tempo de execução serial e paralelo por número de threads da Figura 3, vemos que o tempo serial aumenta linearmente com o aumento das threads, enquanto o tempo paralelo se mantém quase constante. Mais uma métrica que mostra a superioridade de desempenho temporal da versão paralela para a versão serial do programa.

IV. FRACTAL-GENERATION

A. Objetivo

O objetivo desse exercício foi paralelizar o código em C++ que gera o conjunto de Mandelbrot e retorna uma imagem do conjunto em forma de fractal. O conjunto de Mandelbrot é gerado pela iteração de uma recorrência no plano dos números complexos[3]. Foram compilados cinco programas em conjunto, são eles `CycleTimer.h` (cabeçalho), `main.cpp`, `mandelbrotSerial.cpp`, `mandelbrotThread.cpp` e `ppm.cpp`. Gerando o binário `mandelbrot` que quando executado produz duas imagens, `mandelbrot-serial.ppm` gerada de forma serial e `mandelbrot-thread.ppm` gerada de forma paralela.

B. Resultados

A função alterada para implementação de paralelismo foi a `workerThreadStart()` em `mandelbrotThread.cpp`. Foi implementada a divisão de trabalho, fornecendo intervalos de altura da imagem de saída que foram processados por cada thread. A saída do programa apresenta também o tempo de execução serial e paralela com o múltiplo de speedup. Analisando as métricas de desempenho da execução com doze threads, temos que a latência média de execução serial foi de $(0,740 \pm 0,003)$ s. Enquanto a latência média de execução paralela foi de $(0,2 \pm 0,2)$ s. Podemos visualizar a melhora de desempenho com implementação de paralelismo, analisando o gráfico de speedup por threads da Figura 4. Analisando o gráfico de eficiência por threads da Figura 5 notamos o mesmo padrão visto no exercício hello-threads, com a eficiência decrescendo conforme aumenta o número de threads e o custo de paralelização começa gerar um overhead. Olhando para o gráfico de tempo de execução serial e paralelo por threads na Figura 6, vemos a superioridade temporal do programa paralelizado sobre o serial. Diferente do exercício hello-threads, nesse caso o trabalho realizado pelo programa serial permanece fixo com o aumento das iterações, por isso o tempo de execução permanece quase constante. Utilizando a ferramenta gprof integrada com o script `gprof2dot.py`, foi gerada uma representação visual do overhead de cada função do programa, que pode ser vista na Figura 7. O grafo mostra a árvore de execução das funções, cada função é colorizada para representar seu gasto de tempo, verde representa pouco gasto de tempo, azul representa gasto de tempo médio e laranja representa alto gasto de tempo, as cores variam como em um mapa de calor. Podemos confirmar por essa análise, que a função que mais gera overhead no programa é a `mandelbrotSerial()` que executa a parte serial do programa.

V. MONTE-CARLO-PI

A. Objetivo

O objetivo desse exercício foi paralelizar o código em C++ que utiliza o método de Monte Carlo para estimar o valor do número pi. O método de Monte Carlo é um algoritmo computacional que utiliza amostragem aleatória

e repetida para estimar resultados de um evento incerto[4]. Analisando uma forma que consiste em um quadrado de lado igual a dois, contendo dentro de si um círculo de raio igual a um, obtemos a equação $\pi = 4 \cdot \frac{Area\ do\ circulo}{Area\ do\ quadrado}$. Usando o método de Monte Carlo, pode-se gerar uma iteração de pontos aleatórios dentro da região do quadrado, quanto mais pontos obtidos, mais a estimativa se aproxima do valor real de π. Foram compilados os arquivos `monte_carlo_parallel.cpp` e `monte_carlo_serial.cpp` para gerar duas versões de executáveis, `monte_carlo_serial` com execução serial e `monte_carlo_parallel` com execução paralelizada. O programa recebe um número de iterações que irão gerar pontos na região do quadrado, retorna a quantidade de pontos totais e a quantidade de pontos que caíram dentro do círculo.

B. Resultados

A solução foi implementada no arquivo `monte_carlo_parallel.cpp`. A paralelização foi feita dividindo o número de iterações de pontos por threads. A função que mais gera overhead no programa é a `calculate_pi()` que realiza as iterações de pontos. Analisando o desempenho da execução com doze threads e 40.000.000 de iterações, temos que o tempo médio de execução medido com Hyperfine para a versão serial foi de $(4,668 \pm 0,020)$ s. Enquanto que, o tempo médio de execução para a versão paralelizada foi de $(0,586 \pm 0,005)$ s, o speedup medido foi de $(7,97 \pm 0,07)$. Analisando o gráfico de speedup por número de threads na Figura 8, notamos a mesma tendência vista anteriormente, de crescimento do speedup com aumento no número de threads. Porém, nesse caso há uma queda do speedup logo após a marca de doze threads. Esse comportamento pode ser explicado novamente pela geração de concorrência quando o número de threads ultrapassa o número de núcleos do processador (doze núcleos lógicos), causando overhead de escalonamento, contenção de recursos, compartilhamento de cache e outras limitações que podem reduzir o desempenho do programa. O gráfico de eficiência por threads na Figura 9 também expressa essa situação. Olhando para o gráfico de tempo de execução serial e paralelo por threads na Figura 10, observamos o tempo de execução paralela cair e se manter aproximadamente constante com o aumento de threads, enquanto o tempo de execução serial também se mantém aproximadamente constante. Olhando para o gráfico de iterações por threads na Figura 11, vemos que para poucas iterações o tempo de execução serial e paralelo são semelhantes. Enquanto que, para um número alto de iterações, o tempo de execução paralela tende a ser muito menor que o tempo serial. Essa observação reflete os efeitos da granularidade em paralelismo, para tarefas de alta granularidade (mais computação do que comunicação), a divisão de trabalho entre threads compensa o overhead de paralelização.

VI. FALSE-SHARING

A. Objetivo

O objetivo dessa tarefa foi analisar um código paralelizado em que há uma possível ocorrência de false sharing. O evento de false sharing ocorre quando threads compartilham acesso à mesma linha de cache, ao escrever na memória, uma thread modifica o conteúdo que outra estava acessando, causando invalidação de cache. A escrita sucessiva nessa região da memória, causa alta contenção de cache, impactando negativamente no desempenho do programa. O exercício fornece os arquivos `sum_scalar.c` e `sum_scalar_solution.c` que implementam uma soma de escalares, o programa recebe um valor inteiro n e retorna a soma de dois n vezes, com a medição do tempo de execução.

B. Resultados

A solução foi implementada no arquivo `sum_scalar_solution.c`. Primeiramente, foi feita uma análise para verificar a ocorrência de false sharing. Utilizando a ferramenta perf com a opcão c2c, que habilita a análise do compartilhamento de dados de cache para cache. Ao rodar o comando, é gerado um arquivo de saída `perf.data`, com um relatório de estatísticas de cache. Analisando o relatório, olhamos para o campo LLC Misses to Remote cache (HITM), LLC é a sigla para Last Level Cache (L3 geralmente compartilhada entre núcleos) e HITM é a sigla para Hit Modified que faz referência à situação em que um núcleo tenta ler uma linha de cache, mas essa linha está em estado ”Modified”, ou seja, está modificada em outro núcleo. Portanto, o campo citado indica o percentual de vezes que um acesso falhou na cache local e teve que buscar o dado na cache de outro núcleo. Uma porcentagem alta nesse campo é um forte indício para ocorrência de false sharing e foi exatamente o que foi encontrado no exemplo, com 95,3% de resultado para `sum_scalar.c`, como mostra a Figura 12. Podemos analisar também a tabela Shared Data Cache Line Table, que mostra estatísticas detalhadas acerca do compartilhamento de dados em linhas de cache. Olhamos para o campo RmtHitm, que mostra a quantidade de vezes que foram realizados acessos à linhas de cache modificadas, exigindo acesso à cache remota (em outro núcleo). O resultado para `sum_scalar.c` está expresso na Figura 13, confirmando o valor alto do campo RmtHitm[5]. Para solucionar o false sharing foi utilizada a técnica que consiste em adicionar um bloco de bytes entre as variáveis que possam estar causando o problema. O código implementa um vetor de somas parciais para armazenar a contribuição de cada thread na soma final, por essa estrutura ser global e compartilhada entre threads, ela gera fortes indícios de ser a causa do false sharing. Trocando o vetor simples por uma struct com os campos para valor da soma parcial e um vetor de tamanho definido com a intenção de completar 64 bytes totais na estrutura (tamanho da linha de cache), garantimos que cada elemento da soma irá ocupar uma linha de cache, evitando a ocorrência de false sharing. Os novos resultados obtidos após a correção em `sum_scalar_solution.c` podem ser vistos nas Figuras 14 e 15. Também foram gerados gráficos de desempenho para esse exercício, vistos nas Figuras 16, 17 e 18. Nota-se a melhora de desempenho comparando a versão com false sharing e a versão com false sharing corrigido.

APÊNDICE

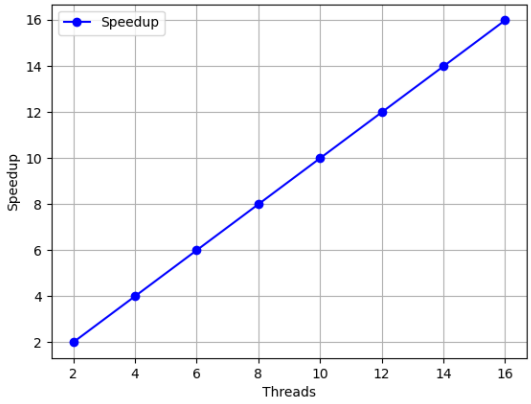


Figura 1: Gráfico de speedup por número de threads no exercício hello-threads

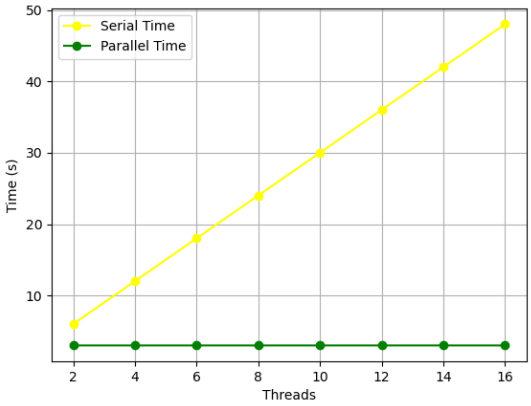


Figura 3: Gráfico de tempo de execução serial e paralelo por número de threads no exercício hello-threads

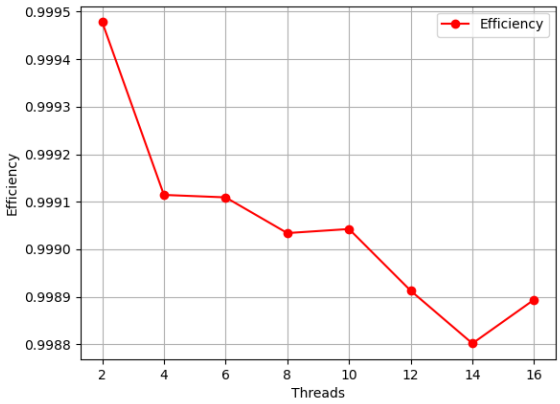


Figura 2: Gráfico de eficiência por número de threads no exercício hello-threads

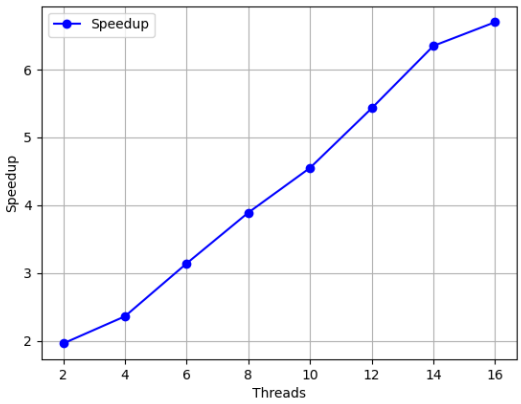


Figura 4: Gráfico de speedup por número de threads no exercício fractal-generation

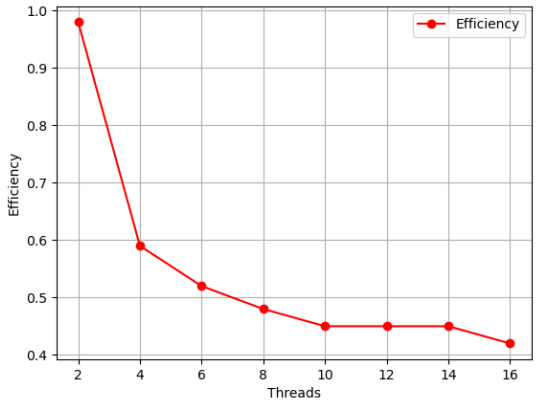


Figura 5: Gráfico de eficiência por número de threads no exercício fractal-generation

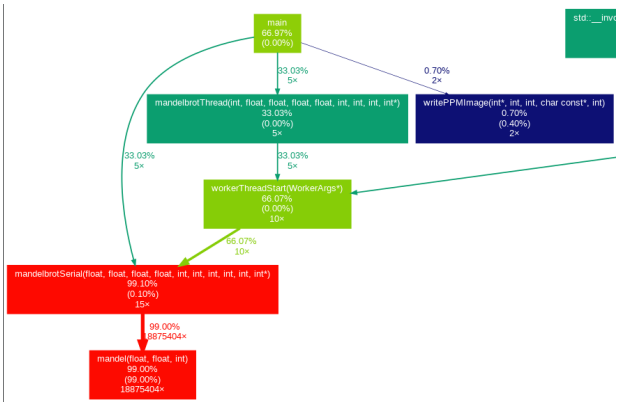


Figura 7: Grafo da árvore de execução das funções e representação de overhead no exercício fractal-generation

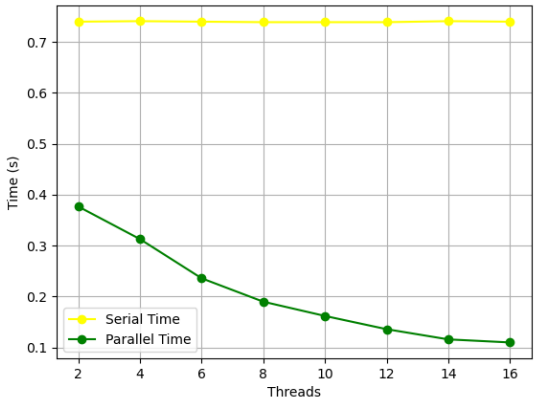


Figura 6: Gráfico de tempo de execução serial e paralelo por número de threads no exercício fractal-generation

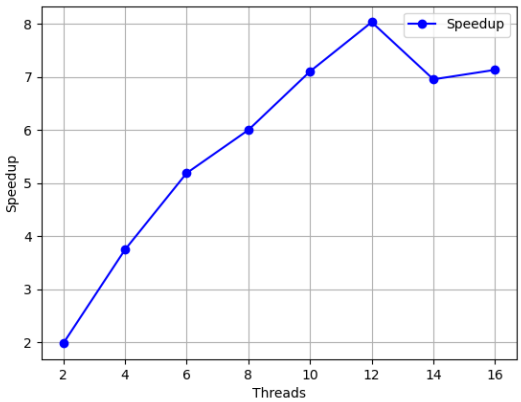


Figura 8: Gráfico de speedup por número de threads no exercício monte-carlo-pi

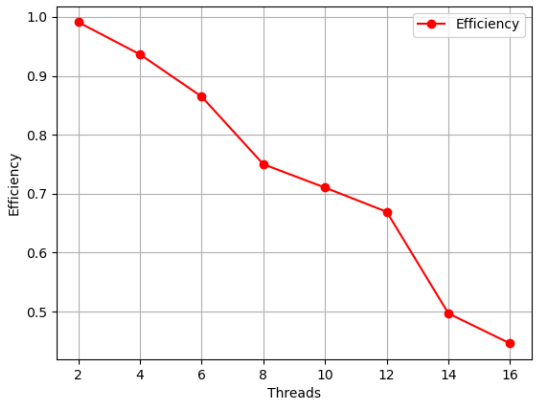


Figura 9: Gráfico de eficiência por número de threads no exercício monte-carlo-pi

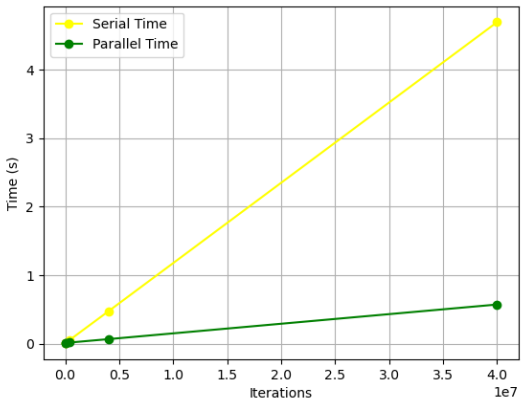


Figura 11: Gráfico de tempo de execução serial e paralelo por número de iterações no exercício monte-carlo-pi

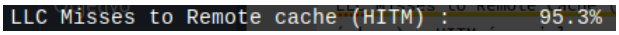


Figura 12: Campo LLC Misses to Remote cache com percentual alto no exercício false-sharing

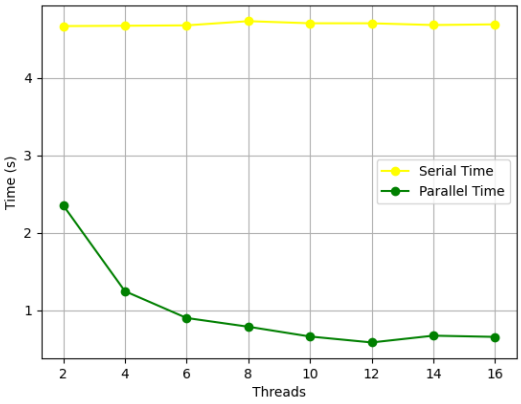


Figura 10: Gráfico de tempo de execução serial e paralelo por número de threads no exercício monte-carlo-pi

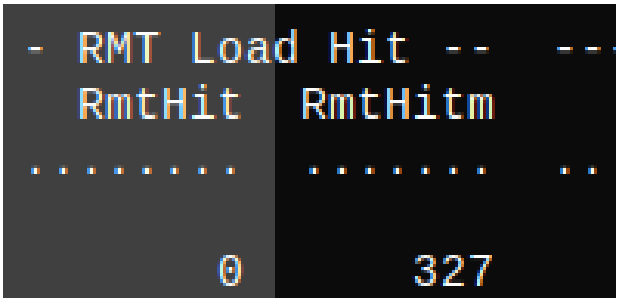


Figura 13: Tabela de compartilhamento de dados em linhas de cache com campo RmtHitm alto no exercício false-sharing

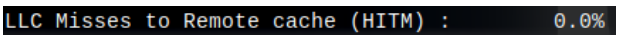


Figura 14: Campo LLC Misses to Remote cache com percentual 0% no exercício false-sharing

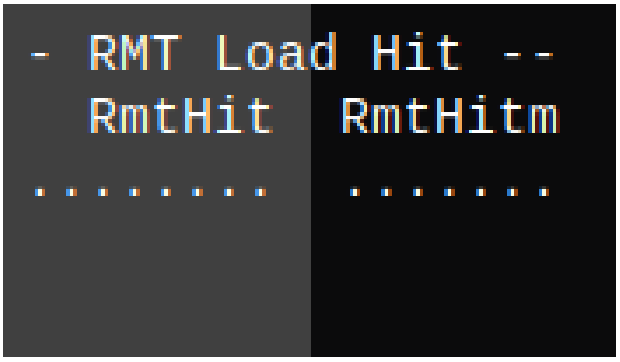


Figura 15: Tabela de compartilhamento de dados em linhas de cache vazia no exercício false-sharing

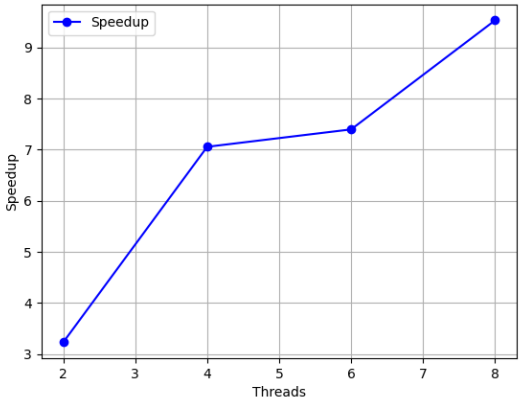


Figura 16: Gráfico de speedup por número de threads no exercício false-sharing

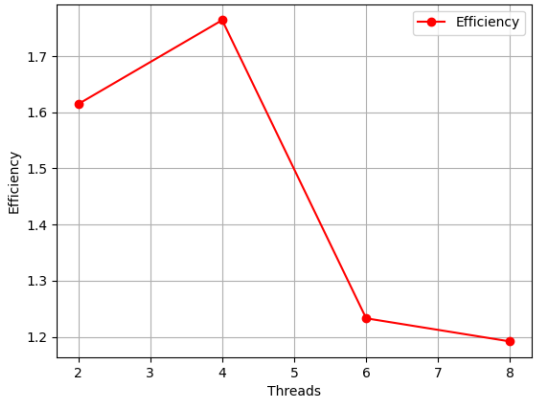


Figura 17: Gráfico de eficiência por número de threads no exercício false-sharing

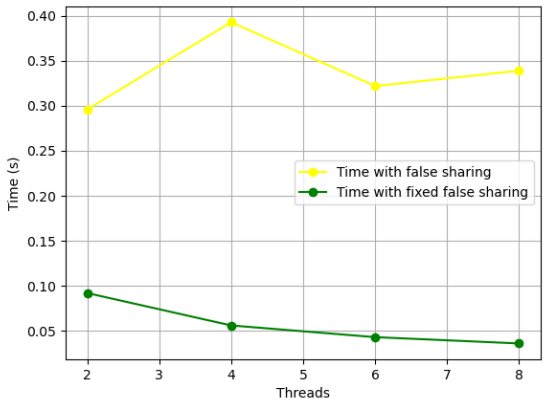


Figura 18: Gráfico de tempo de execução com false sharing e com false sharing corrigido por número de threads no exercício false-sharing

Referências

[1] PETER, D. hyperfine. Disponível em: <https://github.com/sharkdp/hyperfine>.

[2] FONSECA, J. About gprof2dot. Disponível em: <⟨https://github.com/jrfonseca/gprof2dot⟩>.

[3] Fractais. Disponível em: <https://pt.mathigon.org/course/fractals/mandelbrot>.

[4] O que é Simulação de Monte Carlo? — IBM. Disponível em: <https://www.ibm.com/br-pt/topics/monte-carlo-simulation>.

[5] Chapter 26. Detecting false sharing — Red Hat Product Documentation. Disponível em: <https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/monitoring_and_managing_system_status_and_performance/detecting-false-sharing_monitoring-and-managing-system-status-and-performance#detecting-false-sharing-with-perf-c2c_detecting-false-sharing>.